Travaux Pratiques — DevOps

Génie Logiciel / Ingénierie Informatique

Infrastructure as Code & Pipelines CI/CD

Déploiement Automatisé avec Terraform et Docker

Étudiant : Mootez Souilem

Email : mootez.souilem@bysur.com

Date : 16 avril 2026

Outils : Docker, Terraform, GitHub Actions

Durée : 6 heures

Objectif : Maîtriser le Cycle de Vie du Déploiement (DLC) complet via l'IaC
en orchestrant des conteneurs Docker avec Terraform et en automatisant
le processus à travers un pipeline CI/CD GitHub Actions.

Contents

Introduction **3**

1 Partie I : Infrastructure as Code Locale **4**
- 1.1 Structure du Projet . . . 4
- 1.2 Conteneurisation de l'Application (Dockerfile_app) . . . 4
- 1.3 Définition Terraform . . . 5
  - 1.3.1 variables.tf . . . 5
  - 1.3.2 main.tf . . . 5
  - 1.3.3 outputs.tf . . . 7
- 1.4 Exécution du Cycle de Vie du Déploiement (DLC) . . . 7
  - 1.4.1 Étape 1 : Initialisation (`terraform init`) . . . 7
  - 1.4.2 Étape 2 : Planification (`terraform plan`) . . . 8
  - 1.4.3 Étape 3 : Application (`terraform apply`) . . . 8
  - 1.4.4 Étape 4 : Validation . . . 9
  - 1.4.5 Étape 5 : Destruction (`terraform destroy`) . . . 10

2 Partie II : Pipeline CI/CD GitHub Actions **12**
- 2.1 Contexte et Architecture du Pipeline . . . 12
- 2.2 Configuration du Pipeline (`main.yml`) . . . 12
- 2.3 Configuration du Secret GitHub . . . 13
- 2.4 Exécution et Validation du Pipeline . . . 14
  - 2.4.1 Premier déclenchement . . . 14
  - 2.4.2 Validation par modification d'une variable . . . 14

3 Questions d'Approfondissement **16**
- 3.1 Impact de `terraform destroy` sur `terraform.tfstate` . . . 16
- 3.2 Immuabilité de l'Infrastructure . . . 16
- 3.3 Rôle de l'Étape Plan comme Gate du DLC . . . 17
- 3.4 Réseau Docker comme Ressource Terraform . . . 17

Conclusion **19**

List of Figures

1.1 Résultat de `terraform init` — Téléchargement du provider `kreuzwerker/docker` 8
1.2 Résultat de `terraform plan` — Aperçu des 4 ressources à créer 8
1.3 Résultat de `terraform apply` — Création des 4 ressources et affichage des outputs 9
1.4 Accès à `http://localhost:8080` — Page de confirmation du déploiement IaC 10
1.5 Résultat de `docker ps` — Deux conteneurs en état `Up` 10
1.6 Résultat de `terraform destroy` — Suppression des 4 ressources 11

2.1 Architecture du pipeline CI/CD — Flux de déclenchement et étapes DLC . 12
2.2 Configuration du secret `DB_PASSWORD` dans les paramètres GitHub Actions 13
2.3 Exécution réussie du pipeline lors du premier push — Toutes les étapes en vert 14
2.4 Étape Plan du second déclenchement — Détection du changement de variable 14
2.5 Étape Apply du second déclenchement — Mise à jour de la ressource détectée 14

Introduction

L'Infrastructure as Code (IaC) représente une approche fondamentale du DevOps moderne : elle consiste à décrire et gérer l'infrastructure informatique au travers de fichiers de configuration versionnés, lisibles par des humains et interprétables par des machines. Contrairement à une gestion manuelle et ad hoc, l'IaC garantit la **reproductibilité**, la **traçabilité** et la **cohérence** des environnements.

Ce TP s'articule en deux grandes parties :

1. **Partie I** — Déploiement d'une infrastructure locale composée d'un conteneur PostgreSQL et d'une application web Nginx, orchestrés par Terraform via le provider Docker.

2. **Partie II** — Automatisation complète du Cycle de Vie du Déploiement au sein d'un pipeline CI/CD GitHub Actions déclenché à chaque *push* sur la branche principale.

L'environnement de développement utilisé est le suivant :

- Système d'exploitation : Ubuntu 22.04 LTS
- Terraform CLI : version 1.7.x
- Docker Engine : version 24.x
- GitHub Actions : runner `ubuntu-latest`

Chapter 1

Partie I : Infrastructure as Code Locale

1.1 Structure du Projet

Le projet suit une organisation claire et modulaire, où chaque fichier joue un rôle précis dans la chaîne de déploiement :

```
tp-iac-local/
|-- main.tf           (Provider Docker + definition des ressources)
|-- variables.tf      (Parametres configurables : ports, credentials)
|-- outputs.tf        (Informations de sortie apres deploiement)
|-- Dockerfile_app    (Blueprint de l'image Nginx/Alpine)
+-- .gitignore        (Exclusion des fichiers d'etat Terraform)
```

1.2 Conteneurisation de l'Application (Dockerfile_app)

Le `Dockerfile_app` construit une image légère basée sur Alpine Linux, dans laquelle Nginx est installé pour servir une page de validation statique.

```
FROM alpine:latest

RUN apk update && \
    apk add --no-cache nginx

RUN mkdir -p /var/www/localhost && \
    echo "<h1>Application Deployed via Terraform IaC!</h1>" > \
    /var/www/localhost/index.html

RUN mkdir -p /run/nginx

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Listing 1.1: Dockerfile_app — Image Nginx/Alpine

Choix techniques :

- `alpine:latest` : image de base minimaliste (≈ 5 Mo) réduisant la surface d'attaque.

- `-no-cache` : évite la mise en cache du gestionnaire de paquets pour alléger l'image finale.
- `daemon off;` : Nginx s'exécute en premier plan, permettant à Docker de gérer le cycle de vie du processus.
- Création explicite de `/run/nginx` : nécessaire sur Alpine pour que Nginx puisse écrire son PID.

1.3 Définition Terraform

1.3.1 variables.tf

Ce fichier centralise tous les paramètres configurables de l'infrastructure, rendant le code *DRY* (Don't Repeat Yourself) et facilement injectable par le pipeline CI/CD.

```
variable "db_name" {
  description = "Nom de la base de données PostgreSQL."
  type        = string
  default     = "devops_db"
}

variable "db_user" {
  description = "Nom d'utilisateur PostgreSQL."
  type        = string
  default     = "devops_user"
}

variable "db_password" {
  description = "Mot de passe PostgreSQL."
  type        = string
  default     = "strongpassword123"
  sensitive   = true
}

variable "app_port_external" {
  description = "Port externe pour accéder à l'application web."
  type        = number
  default     = 8080
}
```

Listing 1.2: variables.tf — Paramètres configurables

L'attribut `sensitive = true` sur `db_password` empêche Terraform d'afficher la valeur en clair dans les logs du terminal et du pipeline CI/CD.

1.3.2 main.tf

Le fichier central déclare le provider Docker et définit l'ensemble des ressources à provisionner.

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0.1"
```

```
    }
  }
}

provider "docker" {}

resource "docker_image" "postgres_image" {
  name         = "postgres:latest"
  keep_locally = true
}

resource "docker_container" "db_container" {
  name  = "tp-db-postgres"
  image = docker_image.postgres_image.image_id

  ports {
    internal = 5432
    external = 5432
  }

  env = [
    "POSTGRES_USER=${var.db_user}",
    "POSTGRES_PASSWORD=${var.db_password}",
    "POSTGRES_DB=${var.db_name}",
  ]
}

resource "docker_image" "app_image" {
  name = "tp-web-app:latest"

  build {
    context    = path.module
    dockerfile = "Dockerfile_app"
  }

  triggers = {
    dockerfile_hash = filemd5("${path.module}/Dockerfile_app")
  }
}

resource "docker_container" "app_container" {
  name  = "tp-app-web"
  image = docker_image.app_image.image_id

  depends_on = [
    docker_container.db_container
  ]

  ports {
    internal = 80
    external = var.app_port_external
  }
}
```

Listing 1.3: main.tf — Configuration principale Terraform

Points clés du `main.tf` :

- `keep_locally = true` : préserve l'image PostgreSQL localement après `terraform`

`destroy` pour éviter un re-téléchargement inutile.

- `image_id` : attribut correct pour le provider v3.x (l'attribut `latest` a été déprécié).
- `triggers` : force la reconstruction de l'image Docker si le `Dockerfile_app` est modifié.
- `depends_on` : garantit l'ordre de démarrage — la base de données doit être opérationnelle avant le lancement de l'application.

1.3.3 outputs.tf

```
output "db_container_name" {
  description = "Nom du conteneur de la base de données."
  value       = docker_container.db_container.name
}

output "app_access_url" {
  description = "URL d'accès à l'application web."
  value       = "http://localhost:${docker_container.app_container.
   ports[0].external}"
}

output "db_container_id" {
  description = "ID du conteneur PostgreSQL."
  value       = docker_container.db_container.id
}

output "app_container_id" {
  description = "ID du conteneur de l'application web."
  value       = docker_container.app_container.id
}
```

Listing 1.4: outputs.tf — Sorties du déploiement

1.4 Exécution du Cycle de Vie du Déploiement (DLC)

1.4.1 Étape 1 : Initialisation (`terraform init`)

La commande `terraform init` télécharge et installe le provider Docker depuis le registre Terraform.

```
$ cd tp-iac-local
$ terraform init
```

Listing 1.5: Commande d'initialisation

[Capture d'écran à insérer]

Figure 1.1: Résultat de `terraform init` — Téléchargement du provider `kreuzwerker/docker`

Le message `Terraform has been successfully initialized!` confirme que le provider est installé dans le répertoire `.terraform/` et que le fichier de verrou `.terraform.lock.hcl` a été généré.

1.4.2 Étape 2 : Planification (`terraform plan`)

```
$ terraform plan
```

Listing 1.6: Commande de planification

[Capture d'écran à insérer]

Figure 1.2: Résultat de `terraform plan` — Aperçu des 4 ressources à créer

Terraform affiche un plan d'exécution détaillant les **4 ressources** à créer :

1. `docker_image.postgres_image` — Pull de `postgres:latest`
2. `docker_container.db_container` — Conteneur PostgreSQL
3. `docker_image.app_image` — Build de l'image Nginx
4. `docker_container.app_container` — Conteneur de l'application web

Le résumé `Plan: 4 to add, 0 to change, 0 to destroy` confirme qu'aucune infrastructure existante ne sera perturbée.

1.4.3 Étape 3 : Application (`terraform apply`)

```
$ terraform apply -auto-approve
```

Listing 1.7: Commande d'application

```
      + ports {
          + external = 8081
          + internal = 80
          + ip       = "0.0.0.0"
          + protocol = "tcp"
        }
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  ~ app_access_url    = "http://localhost:8080" -> "http://localhost:8081"
  + app_container_id  = (known after apply)
docker_container.app_container: Creating...
docker_container.app_container: Creation complete after 0s [id=fd48eecbd07be9c80658211aef479354bc12c8e4f16f2b70
]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

app_access_url = "http://localhost:8081"
app_container_id = "fd48eecbd07be9c80658211aef479354bc12c8e4f16f2b7057bd3da77a8d977c"
db_container_id = "89ec06a01586152fa61f6b8839b56b4a6e50ec0a6b40b5f351cb5f0ac9dbae75"
db_container_name = "tp-db-postgres"
mootez@bysur:~/Desktop/devops/tp3/tp-iac-local$
to download.
```

Figure 1.3: Résultat de `terraform apply` — Création des 4 ressources et affichage des outputs

À l'issue de l'apply, Terraform affiche les *outputs* définis dans `outputs.tf`, notamment l'URL d'accès :

```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:
app_access_url    = "http://localhost:8080"
db_container_name = "tp-db-postgres"
```

1.4.4 Étape 4 : Validation

Après l'application, la vérification s'effectue à deux niveaux :

Vérification via navigateur

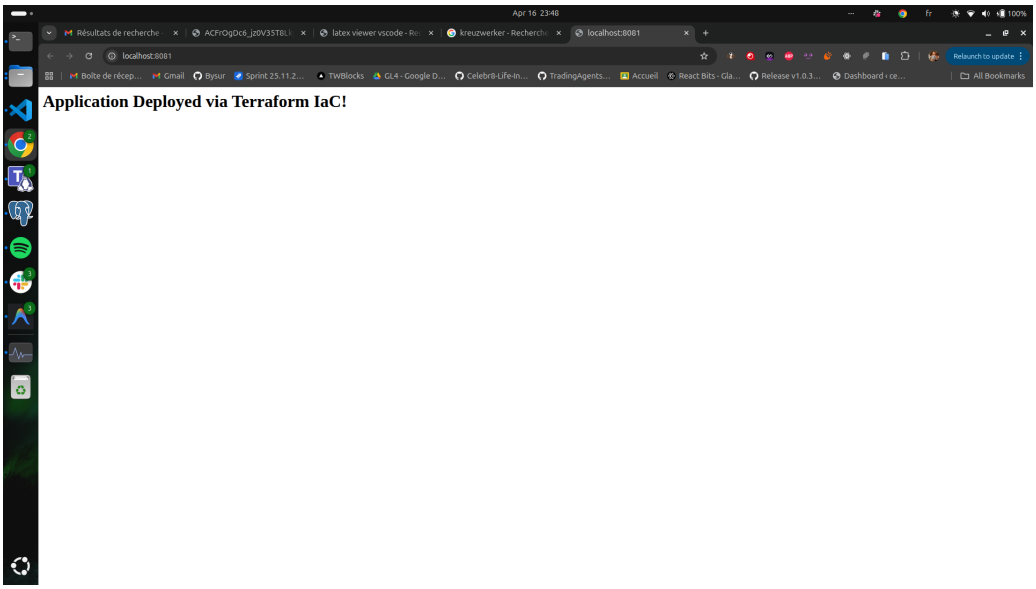

Figure 1.4: Accès à `http://localhost:8080` — Page de confirmation du déploiement IaC

Vérification via Docker CLI

```
$ docker ps
```

Listing 1.8: Vérification de l'état des conteneurs

Figure 1.5: Résultat de `docker ps` — Deux conteneurs en état `Up`

Les deux conteneurs `tp-db-postgres` et `tp-app-web` apparaissent avec le statut `Up`, confirmant le bon déroulement du déploiement.

1.4.5 Étape 5 : Destruction (`terraform destroy`)

```
$ terraform destroy -auto-approve
```

Listing 1.9: Commande de destruction

```
  # docker_image.postgres_image will be destroyed
  - resource "docker_image" "postgres_image" {
      - id           = "sha256:413d57cb67bb3585acae8769a48661aa1dcac66076e3a5f88ea6c5be55f8916epostgres:latest" -> null
      - image_id     = "sha256:413d57cb67bb3585acae8769a48661aa1dcac66076e3a5f88ea6c5be55f8916e" -> null
      - keep_locally = true -> null
      - name         = "postgres:latest" -> null
      - repo_digest  = "postgres@sha256:52e6ffd11fddd081ae63880b635b2a61c14008c17fc98cdc7ce5472265516dd0" -> null
    }

Plan: 0 to add, 0 to change, 4 to destroy.

Changes to Outputs:
  - app_access_url    = "http://localhost:8081" -> null
  - app_container_id  = "52b3d8a8476b425ebf117531aafcb3113e57f2f15c5b02b8ceb063762859ac7a" -> null
  - db_container_id   = "89ec06a01586152fa61f6b8839b56b4a6e50ec0a6b40b5f351cb5f0ac9dbae75" -> null
  - db_container_name = "tp-db-postgres" -> null
docker_container.app_container: Destroying... [id=52b3d8a8476b425ebf117531aafcb3113e57f2f15c5b02b8ceb063762859ac7a]
docker_container.app_container: Destruction complete after 0s
docker_image.app_image: Destroying... [id=sha256:09022687aabfdcbde371b92e20dda421358b2cc48c624a7c74eed11108ecdcf6tp-web-app:l
est]
docker_container.db_container: Destroying... [id=89ec06a01586152fa61f6b8839b56b4a6e50ec0a6b40b5f351cb5f0ac9dbae75]
docker_image.app_image: Destruction complete after 0s
docker_container.db_container: Destruction complete after 0s
docker_image.postgres_image: Destroying... [id=sha256:413d57cb67bb3585acae8769a48661aa1dcac66076e3a5f88ea6c5be55f8916epostgre
latest]
docker_image.postgres_image: Destruction complete after 0s

Destroy complete! Resources: 4 destroyed.
```

Figure 1.6: Résultat de `terraform destroy` — Suppression des 4 ressources

Le message final `Destroy complete! Resources: 4 destroyed.` confirme la suppression propre de l'ensemble des ressources provisionnées.

Chapter 2

Partie II : Pipeline CI/CD GitHub Actions

2.1 Contexte et Architecture du Pipeline

L'objectif de cette partie est d'automatiser le Cycle de Vie du Déploiement en intégrant les commandes Terraform dans un pipeline GitHub Actions. Ce pipeline se déclenche automatiquement à chaque *push* sur la branche `main`.

[Capture d'écran à insérer]

Figure 2.1: Architecture du pipeline CI/CD — Flux de déclenchement et étapes DLC

2.2 Configuration du Pipeline (`main.yml`)

```yaml
name: Terraform IaC Pipeline

on:
  push:
    branches:
      - main

jobs:
  terraform:
    name: Deploy Infrastructure
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: tp-iac-local

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

```

```yaml
      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: "1.7.0"

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        id: plan
        run: terraform plan -var="db_password=${{ secrets.DB_PASSWORD }}" -out=tfplan
        continue-on-error: false

      - name: Terraform Apply
        if: steps.plan.outcome == 'success'
        run: terraform apply -auto-approve tfplan
```

Listing 2.1: .github/workflows/main.yml — Pipeline GitHub Actions

Points de conception importants :

- `working-directory: tp-iac-local` : les commandes Terraform s'exécutent dans le bon répertoire sans nécessiter de `cd` explicite à chaque étape.
- `continue-on-error: false` sur l'étape Plan : tout échec du plan arrête immédiatement le pipeline, empêchant un apply sur une base défaillante.
- `if: steps.plan.outcome == 'success'` : la condition de garde (*gate*) garantit que l'étape Apply ne s'exécute que si et seulement si le Plan a réussi.
- `secrets.DB_PASSWORD` : le mot de passe de la base de données est injecté via les secrets GitHub, sans jamais apparaître en clair dans le code source.
- `-out=tfplan` : le plan est sauvegardé dans un fichier binaire, que l'étape Apply consomme directement — garantissant que l'état appliqué correspond exactement à ce qui a été planifié.

2.3 Configuration du Secret GitHub

Avant de pousser le projet, le secret `DB_PASSWORD` doit être créé dans les paramètres du dépôt GitHub : `Settings > Secrets and variables > Actions > New repository secret`.

[Capture d'écran à insérer]

Figure 2.2: Configuration du secret `DB_PASSWORD` dans les paramètres GitHub Actions

2.4 Exécution et Validation du Pipeline

2.4.1 Premier déclenchement

```
$ git init
$ git add .
$ git commit -m "feat: initial IaC setup with Terraform and Docker"
$ git remote add origin https://github.com/<username>/tp-iac-local.git
$ git push -u origin main
```

Listing 2.2: Push initial vers la branche main

[Capture d'écran à insérer]

Figure 2.3: Exécution réussie du pipeline lors du premier push — Toutes les étapes en vert

2.4.2 Validation par modification d'une variable

Conformément aux consignes, le mot de passe de la base de données a été modifié dans `variables.tf` pour déclencher un second cycle de déploiement.

```
$ git add variables.tf
$ git commit -m "chore: update db_password default value"
$ git push origin main
```

Listing 2.3: Modification et push pour valider la mise à jour

[Capture d'écran à insérer]

Figure 2.4: Étape Plan du second déclenchement — Détection du changement de variable

[Capture d'écran à insérer]

Figure 2.5: Étape Apply du second déclenchement — Mise à jour de la ressource détectée

Le pipeline détecte le changement de variable dans l'étape Plan (`1 to change`) et applique la modification lors de l'étape Apply, démontrant la boucle de réconciliation automatique de Terraform.

Chapter 3

Questions d'Approfondissement

3.1 Impact de `terraform destroy` sur `terraform.tfstate`

`terraform.tfstate` est le **fichier d'état** de Terraform : il contient une représentation JSON de l'infrastructure réelle telle que Terraform la connaît, incluant les identifiants des ressources créées, leurs attributs actuels, et les dépendances entre elles.

Lors de l'exécution de `terraform destroy` :

1. Terraform lit le `terraform.tfstate` pour identifier toutes les ressources à supprimer et leur ordre de destruction (inversé par rapport à la création, en respectant les dépendances).
2. Il contacte le provider (Docker dans notre cas) pour effectivement supprimer les ressources.
3. Il met à jour le `terraform.tfstate` : les ressources supprimées sont retirées, laissant un fichier d'état vide (ou ne contenant que les ressources non détruites).

Rôle dans la réconciliation : Le fichier d'état est le pont entre *l'infrastructure déclarée dans le code* et *l'infrastructure réelle*. À chaque `plan` ou `apply`, Terraform compare l'état désiré (les fichiers `.tf`) à l'état connu (le `.tfstate`) et à l'état réel (via des appels API au provider) pour calculer le diff minimal à appliquer. Sans ce fichier, Terraform ne saurait pas quelles ressources il a déjà créées et tenterait de tout recréer à chaque exécution.

Attention en production

En production, le fichier `terraform.tfstate` doit être stocké dans un **backend distant** (S3, Azure Blob, Terraform Cloud) avec verrouillage (*state locking*) pour éviter les corruptions en cas d'accès concurrent. Ne jamais committer ce fichier dans Git.

3.2 Immuabilité de l'Infrastructure

L'immuabilité de l'infrastructure (*Immutable Infrastructure*) est un principe selon lequel **un serveur ou un conteneur déployé ne doit jamais être modifié après sa création**. Toute modification passe par la création d'une nouvelle instance qui remplace l'ancienne, au lieu de muter l'instance existante.

Approche Mutable	Approche Immuable
On se connecte au serveur en SSH et on met à jour le paquet Nginx	On reconstruit une nouvelle image Docker avec la nouvelle version, on déploie le nouveau conteneur, on supprime l'ancien
Risque de *configuration drift*	Infrastructure toujours dans un état connu et reproductible
Retour arrière difficile	Retour arrière = redéployer l'image précédente

Dans notre TP, l'utilisation du mécanisme `triggers` dans la ressource `docker_image.app_image` illustre ce principe : toute modification du `Dockerfile_app` provoque la **reconstruction complète** de l'image et le **remplacement** du conteneur, jamais sa modification en place.

3.3 Rôle de l'Étape Plan comme Gate du DLC

L'étape `terraform plan` joue le rôle de **porte de validation** (*quality gate*) essentielle dans le DLC pour plusieurs raisons :

1. **Prévisibilité sans risque** : le plan simule l'intégralité des changements sans toucher à l'infrastructure réelle. Un opérateur peut vérifier exactement ce qui sera modifié, créé ou supprimé.
2. **Détection précoce d'erreurs** : les erreurs de syntaxe, de références incorrectes ou de conflits de ressources sont détectées avant toute modification de production.
3. **Séparation des responsabilités** : dans un contexte d'équipe, le plan peut être soumis à une revue humaine (*plan review*) ou à une approbation manuelle avant l'apply — c'est le principe du *four-eyes principle*.
4. **Intégrité du pipeline** : dans notre fichier `main.yml`, la condition `if: steps.plan.outcome == 'success'` garantit que l'étape Apply n'est jamais exécutée sur une base de plan défaillante.
5. **Auditabilité** : le fichier `tfplan` généré constitue une preuve immuable de ce qui était prévu d'être appliqué, utilisable à des fins d'audit.

3.4 Réseau Docker comme Ressource Terraform

Pour créer un réseau Docker dédié et y connecter les deux conteneurs, la ressource Terraform à utiliser est **`docker_network`** du provider `kreuzwerker/docker`. Voici comment l'intégrer dans `main.tf` :

```
resource "docker_network" "app_network" {
  name = "tp-app-network"
}

resource "docker_container" "db_container" {
```

```
  name  = "tp-db-postgres"
  image = docker_image.postgres_image.image_id

  networks_advanced {
    name = docker_network.app_network.name
  }

  env = [
    "POSTGRES_USER=${var.db_user}",
    "POSTGRES_PASSWORD=${var.db_password}",
    "POSTGRES_DB=${var.db_name}",
  ]
}

resource "docker_container" "app_container" {
  name  = "tp-app-web"
  image = docker_image.app_image.image_id

  depends_on = [docker_container.db_container]

  networks_advanced {
    name = docker_network.app_network.name
  }

  ports {
    internal = 80
    external = var.app_port_external
  }
}
```

Listing 3.1: Ajout d'un réseau Docker partagé

Avantages de cette approche :

- Les conteneurs communiquent par leur nom DNS (ex : le conteneur `app` peut joindre le conteneur `db` via l'hôte `tp-db-postgres`).
- Le port 5432 de PostgreSQL n'a plus besoin d'être exposé sur l'hôte, améliorant la sécurité (*principle of least exposure*).
- L'infrastructure est plus proche d'un environnement de production réel.

Conclusion

Ce TP a permis de mettre en pratique l'ensemble du cycle de vie d'un déploiement infrastructure moderne, depuis la définition déclarative des ressources avec Terraform jusqu'à leur automatisation complète via un pipeline CI/CD GitHub Actions.

Les compétences acquises couvrent trois dimensions complémentaires :

- **Technique** : maîtrise du provider Docker Terraform (v3.x), gestion des dépendances entre ressources, build d'images Docker depuis Terraform, gestion des outputs et des variables sensibles.
- **Processus** : compréhension du Cycle de Vie du Déploiement à travers ses cinq étapes (Init → Plan → Apply → Validate → Destroy), avec la distinction cruciale entre phase de planification (prévisualisation sans risque) et phase d'application (mutation réelle).
- **Culture DevOps** : appréhension des principes fondateurs — immuabilité de l'infrastructure, gestion de l'état, séparation entre code d'infrastructure et secrets, et automatisation comme garant de la cohérence.

Ce socle constitue une base solide pour aborder des environnements cloud réels (AWS, Azure, GCP) où Terraform est l'outil de facto pour l'orchestration d'infrastructure à grande échelle.